

Low Level Inference Optimization (Individual contribution)

Hrishikesh Gogoi

Feb 2025 - May 2025

1	Project summary	2
2	Context and Problem statement	2
3	Target environment	3
4	My contributions	3
5	Performance graphs (before tiling)	4
6	Definitions	5
7	Architecture	6
8	Example	8
9	Illustration	9
10	Performance graphs (using tiling)	12
11	Results and key findings	13
12	Challenges and debugging	13

1 Project summary

Optimized low-level compute kernels for dense matrix multiplication routines used in neural network inference on the target hardware. Replaced naive memory access patterns with hardware-aware tiling, achieving an average $1.47\times$ speedup across the tested matrix dimensions.

2 Context and Problem statement

General Matrix Multiply (GEMM) operations form the primary computational bottleneck in inference workloads. Standard naive implementations trigger high global memory traffic, poor cache locality and underutilized execution units (EUs) on the target hardware. This project addresses these constraints by implementing a custom-tailored tiled execution pipeline to optimize memory reuse and maximize compute throughput.

3 Target environment

Hardware:

- Host Platform: Khadas Mind Maker Kit
- Processor (CPU): Intel(R) Core(TM) Ultra 7 258V
- Target Accelerator (GPU): Intel(R) Arc(TM) Graphics

Software:

- Operating System: Ubuntu 24.10
- Linux Kernel: 6.11.0-21-generic
- Programming Model: SYCL 2020 (DPC++ Runtime)

4 My contributions

- Benchmarked matrix multiplication across different matrix dimensions and sizes.
- Investigated GPU compatibility and runtime issues.
- Analyzed and implemented tiled matrix multiplication.
- Implemented `std::chrono` benchmarking with explicit queue synchronization to measure GPU execution time.
- Analyzed work-item and workgroup mapping to Xe2 execution resources and optimized memory-aware tiling.

5 Performance graphs (before tiling)

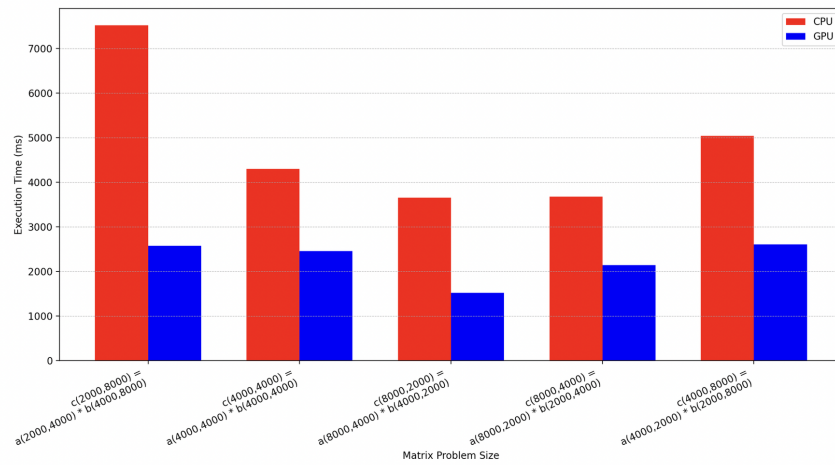


Figure 1: Performance on different matrix shapes

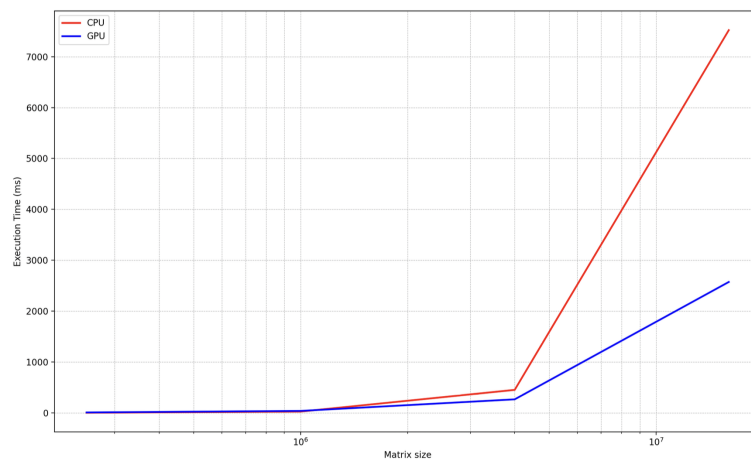


Figure 2: Performance on different matrix sizes

6 Definitions

Work-item = A work-item is the smallest unit of execution in a parallel program

Work-group = A work-group is a collection of work-items that execute together and can share local memory

Sub-group = A sub-group is a smaller collection of work-items inside a work-group, that can operate together

Xe Core = It is a fundamental compute unit in Intel's newer GPU architectures

XVE(Xe Vector Engine) = It is responsible for executing vector operations.

SIMD lane = It is a single processing unit within an XVE (Xe Vector Engine) that executes the computation for one position in a vector operation, processing a set of operands ($A \times B + C$) from that position. Depending on the data type, such as FP16, it may perform multiple operations per cycle.

XMV(Xe Matrix eXtensions) = They are specialized hardware units within each Xe core designed to accelerate matrix operations
MAC = It refers to the Multiply-Accumulate operation.

Ops/MAC(Operations per MAC) = A MAC here is a fused multiply-add(one multiply plus one add in a single cycle)

Depth = Pipeline stages per XMV engine, allowing data to flow through chained MAC stages to hide latency

7 Architecture

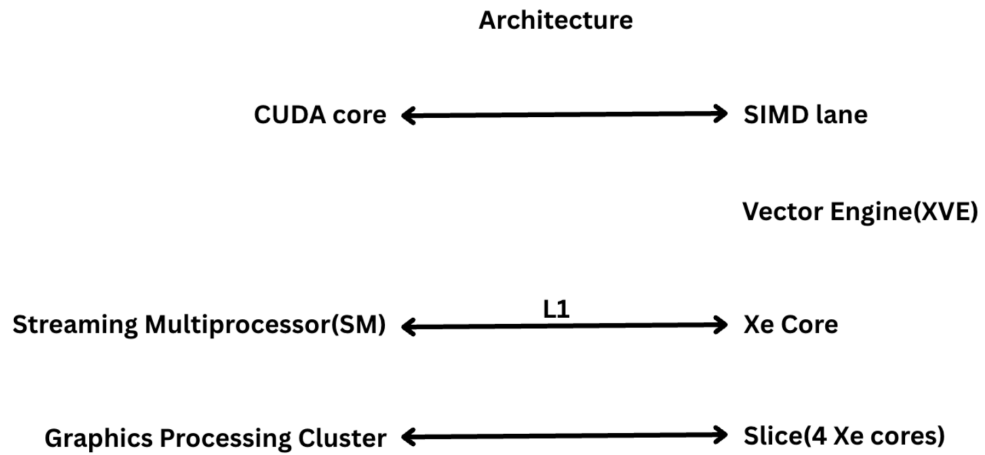


Figure 3: NVIDIA to Intel

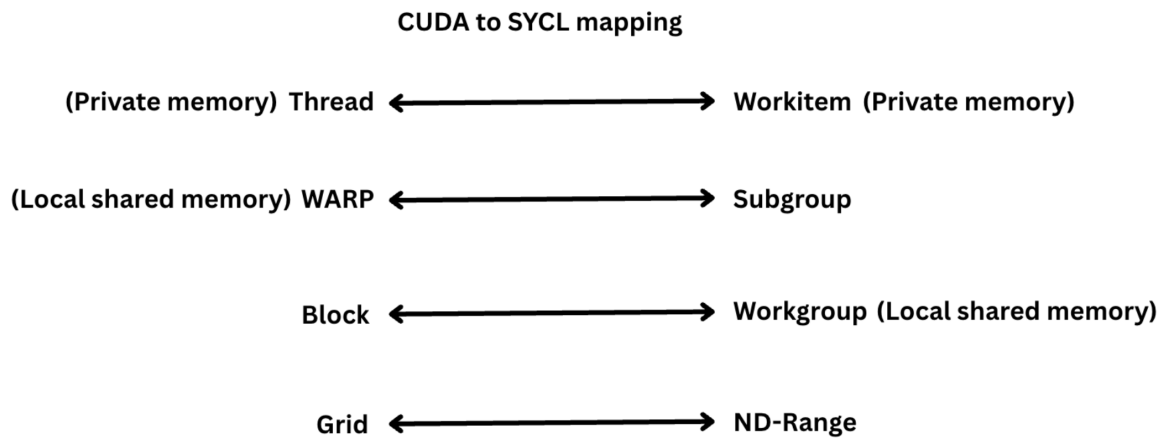


Figure 4: CUDA to SYCL

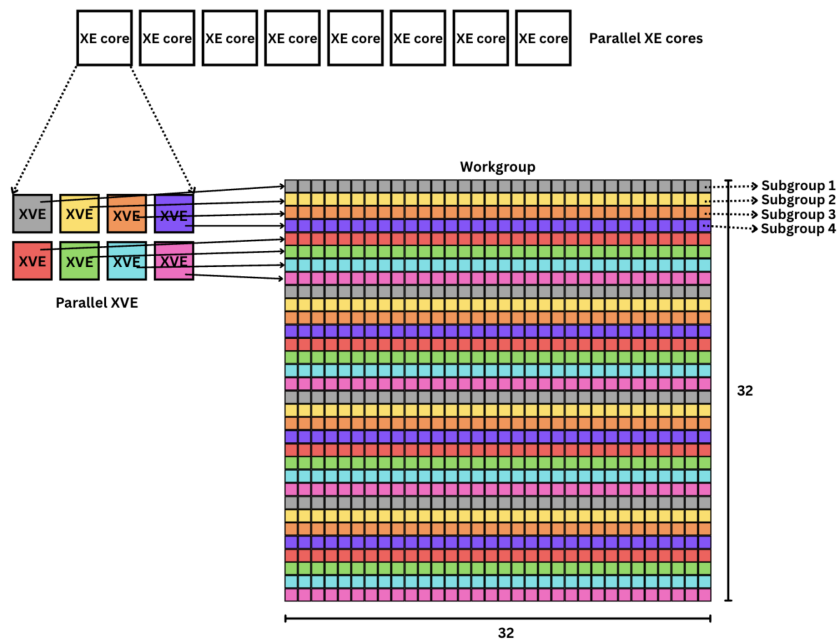


Figure 5: Architecture

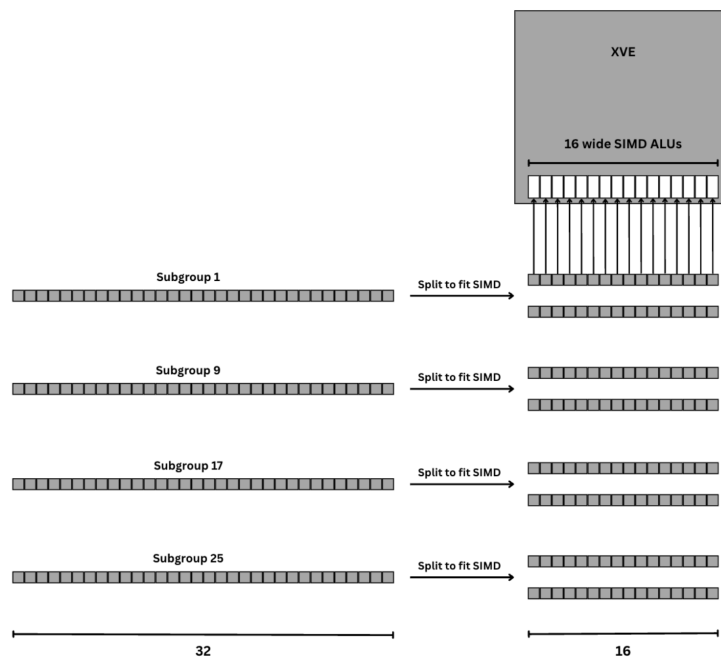


Figure 6: XVE

8 Example

For matrix multiplication of matrices $A_{m \times k}$ and $B_{k \times n}$ yielding $C_{m \times n}$

$$A_{m \times k} \times B_{k \times n} = C_{m \times n}$$

The total number of required Fused Multiply-Add (FMA) operations is

$$N_{\text{FMA}} = m \times k \times n$$

Consider square matrices where $m = k = n = 512$:

$$N_{\text{total_FMA}} = 512 \times 512 \times 512 = 134,217,728 \text{ operations}$$

To match hardware resource constraints (maximum workgroup size of 1024), matrices are partitioned into sub-blocks of size 32×32 :

$$A_{512 \times 512} = [X]_{16 \times 16}, \quad B_{512 \times 512} = [Y]_{16 \times 16}$$

where X, Y are sub-blocks of size 32×32 .

Number of sub-block operations required:

$$N_{\text{sub-block_ops}} = 16 \times 16 \times 16 \text{ sub-block operations}$$

Distributing the workload across 8 cores:

$$\text{Operations per core} = 16 \times 16 \times 2 \text{ sub-block operations} \quad \text{---(R1)}$$

Note: For efficient multiplication, the pair of sub-blocks corresponding to the i -th row and j -th column are mapped to the same core.

For a single sub-block = $32 \times 32 \times 32$ FMA operations

Dividing it among 8 XVE = $32 \times 32 \times 4$ FMA operations

Dividing it among 16 SIMD lanes = $32 \times 2 \times 4$ FMA operations

256 cycles of SIMD are required per sub-block

For all sub-blocks, total SIMD cycles = $256 \times (16 \times 16 \times 2) = 131,072$ SIMD cycles

9 Illustration

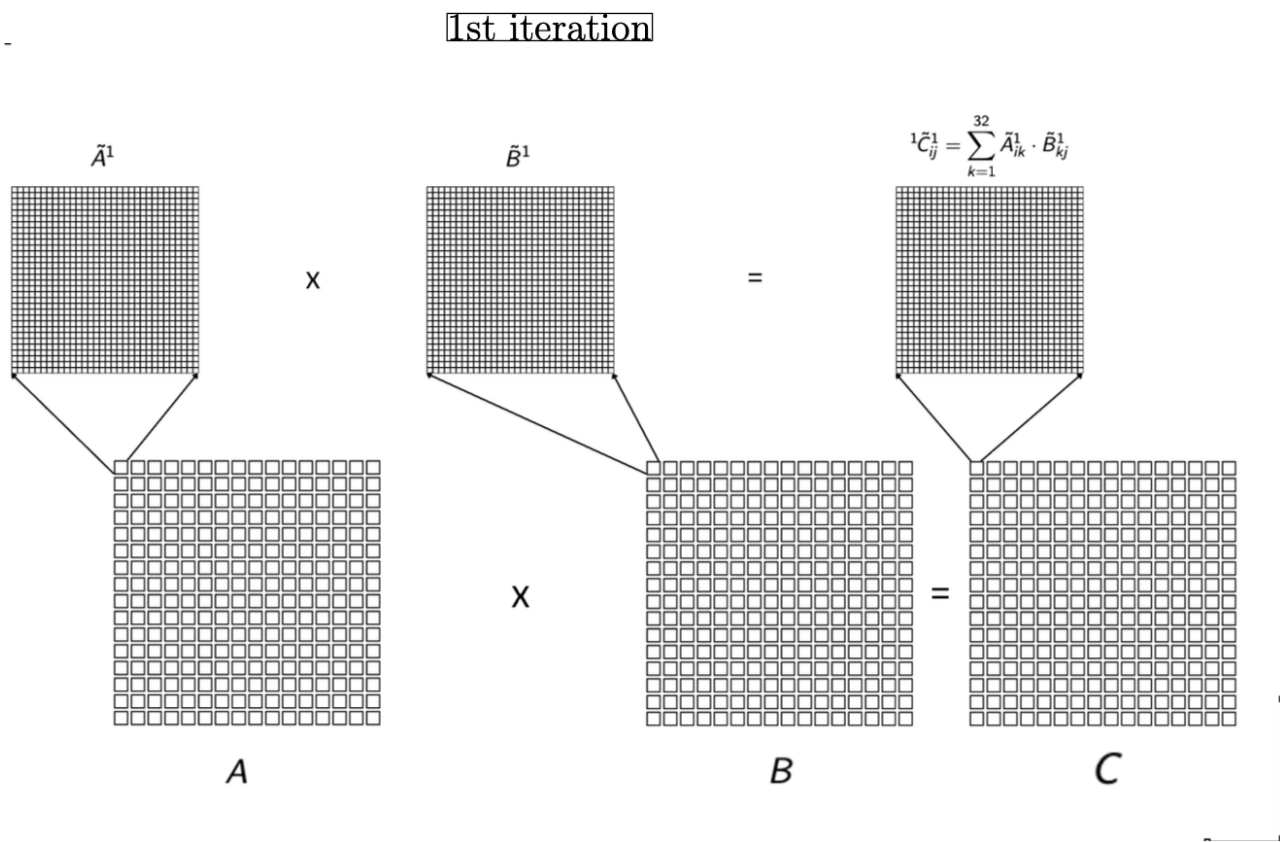
${}^t\tilde{C}_{ij}^k$ is the k -th tile of the matrix C in row-major order

where,

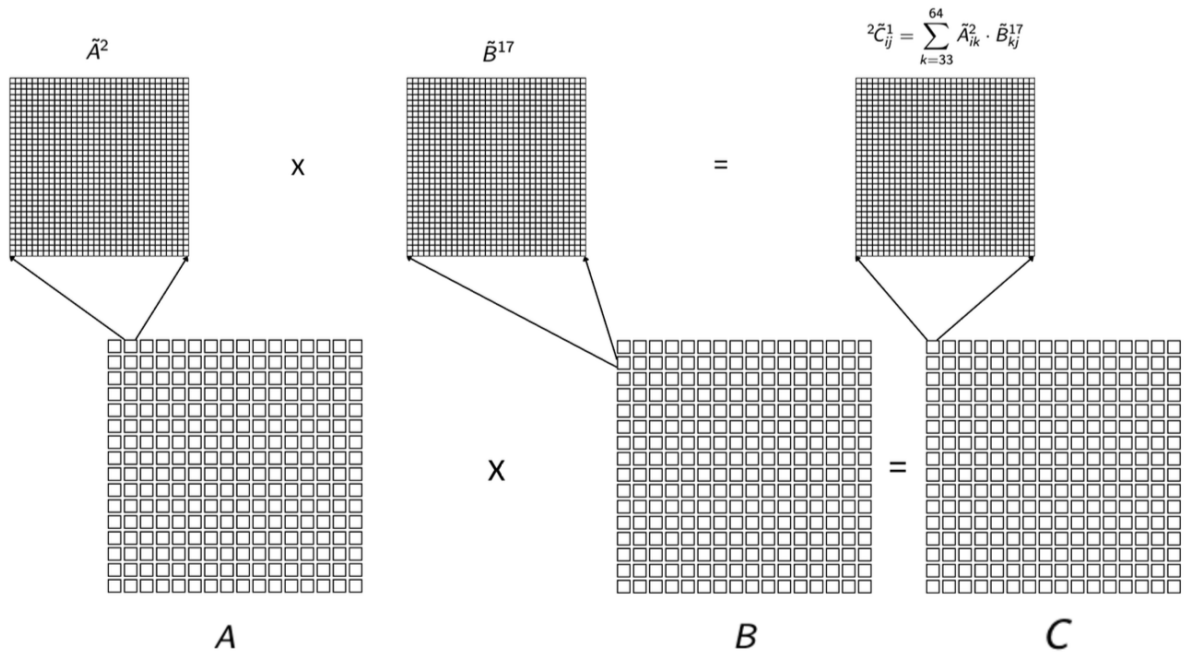
t = iteration

i = row

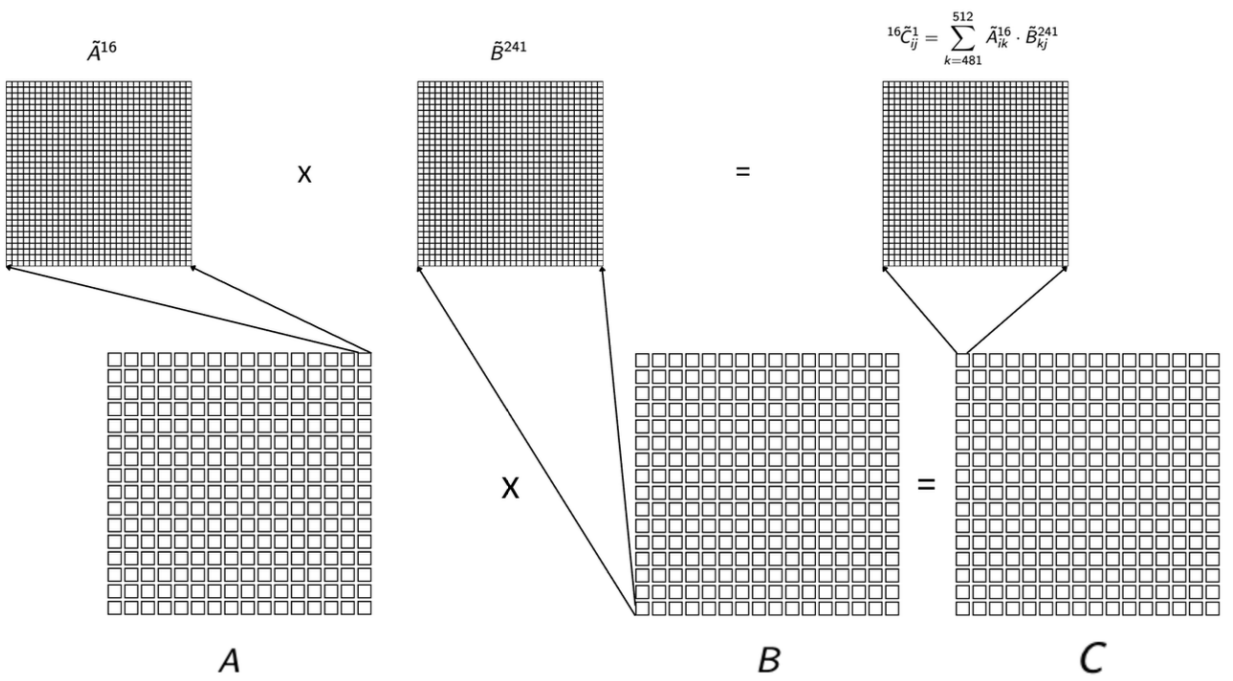
j = column

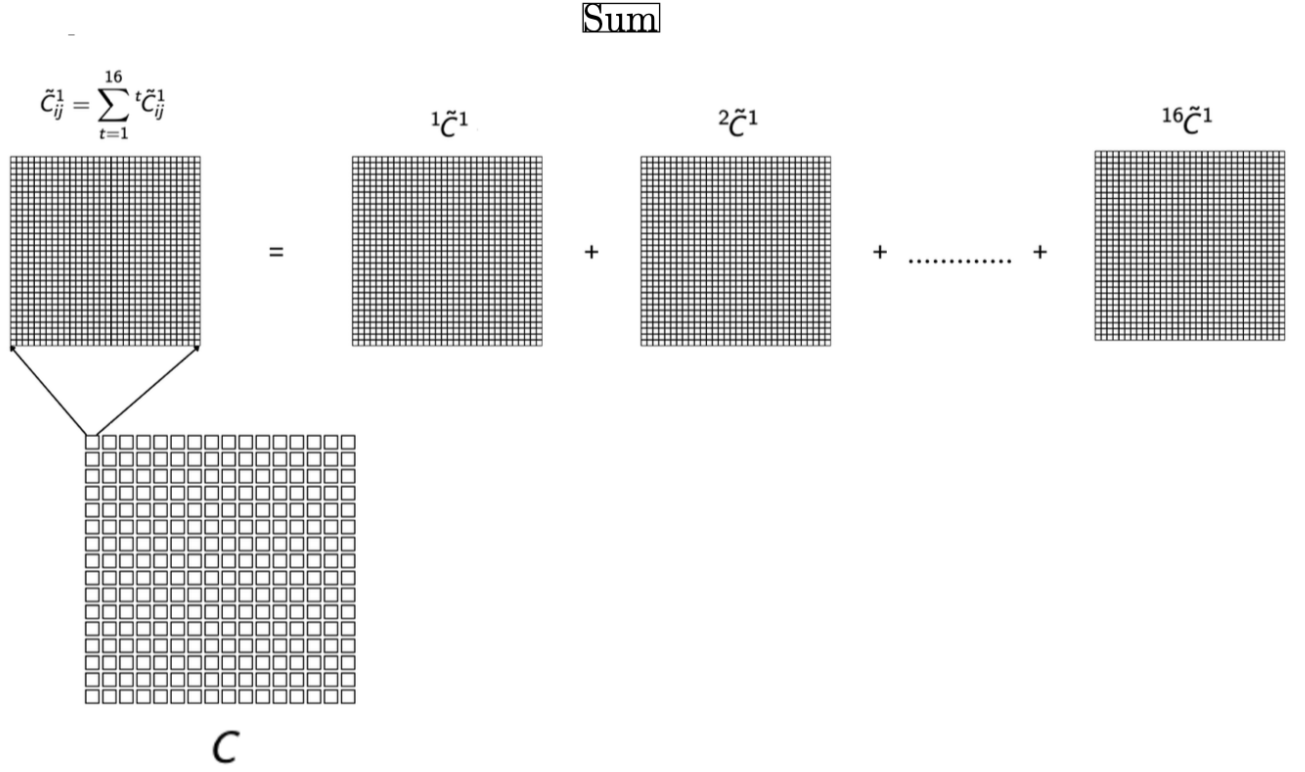


2nd iteration



nth iteration





In each ${}^k\tilde{C}_{ij}^1$

32 * 32 workgroup is divided among 8 XVE, so each XVE handles 128 workitems

or 128 * 32 FMA(32 Multiply + 31 Accumulate) = 4096 MAC operations per XVE

For $\tilde{C}_{ij}^1$

There are 16 such iterations:

$$16 * 4096 = 65536 \text{ FMA operations per XVE}$$

For C

There are 256 such workgroups divided among 8 XE cores so each XE core handles 32 workgroups

$$32 * 65536 = 2097152 \text{ FMA operations per XVE}$$

There are 16 SIMD lanes which can run 16 FMA operations parallelly

$$2097152/16 = 131072 \text{ cycles in total}$$

10 Performance graphs (using tiling)

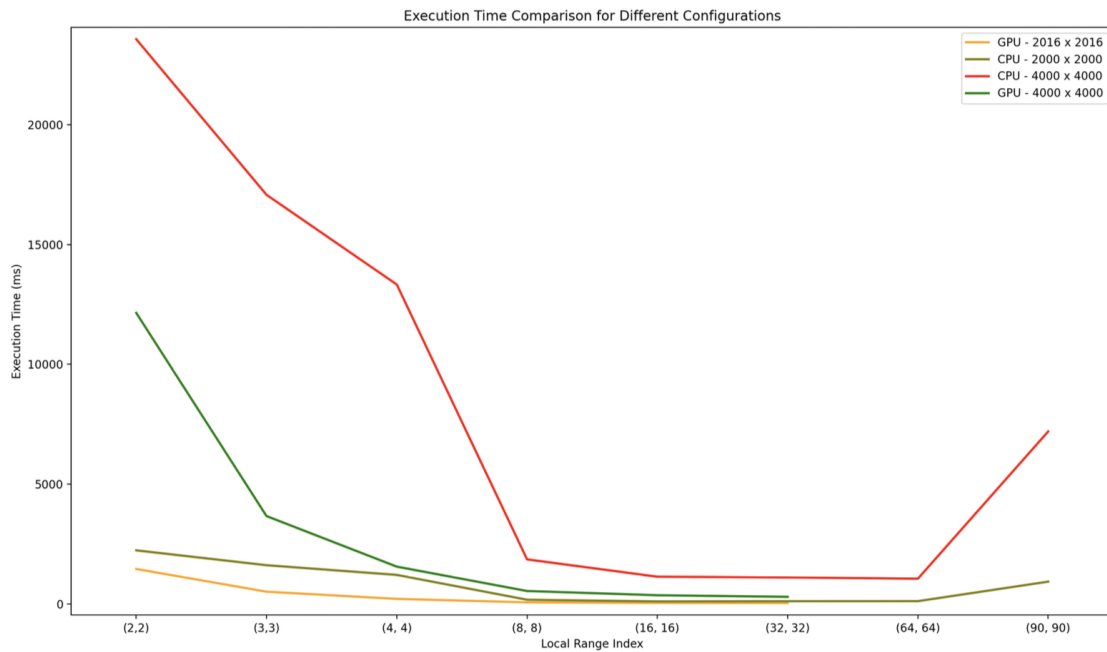


Figure 7: Multiplication Performance on different workgroup size

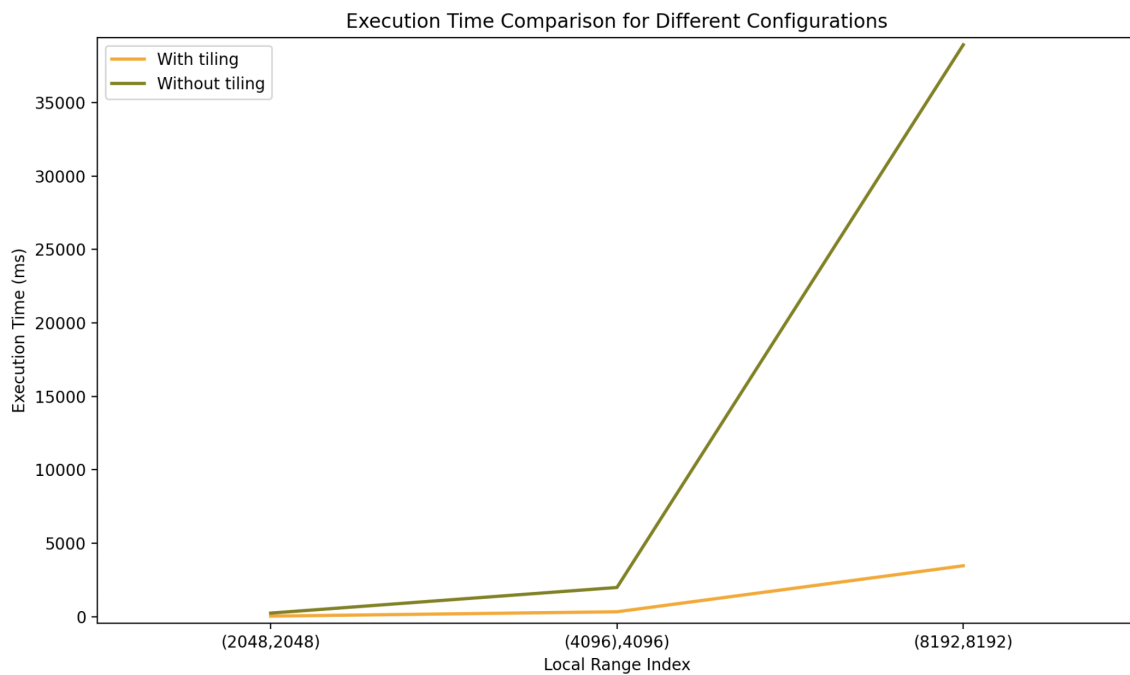


Figure 8: Tiling vs no tiling

11 Results and key findings

- Different matrix shapes and sizes produced different execution characteristics.
- Tiling improved memory locality and matrix multiplication performance.
- Work-item/workgroup mapping to Xe2 execution resources provided insight into the computation distribution across XE cores, XVEs and SIMD lanes.
- Hardware-aware tile sizing was a key performance driver, scaling data reuse while remaining within the available SLM(Shared Local Memory) capacity.
- Achieved an average $1.47\times$ speedup across the tested matrix dimensions.

12 Challenges and debugging

- Resolved GPU driver configuration issues to enable oneAPI runtime execution.
- Applied zero-padding to matrices A and B to ensure matrix dimensions were evenly divisible by the work-group size, satisfying SYCL runtime *nd_range* execution requirements.
- VTune was not functioning correctly. Modified “kernel.kptr_restrict” to 0 to resolve issues with accessing kernel symbols, which was preventing vtune from retrieving necessary information. Vtune was able to access CPU but not GPU, so opted for SYCL’s explicit queue synchronization (`q.wait()`) coupled with `std::chrono` high-resolution clocks. This architectural barrier forces the CPU to wait for the hardware to finish, precisely isolating and measuring the execution duration of the GPU kernel.

END